

TWO POINTERS • SLIDING WINDOW • PREFIX SUM

Detailed Algorithm Comparison — Textbook Edition

Google DSA Interview Preparation

The complete algorithm-comparison conversation, converted into a structured textbook-style PDF without dropping a word.

Core distinction

Two Pointers = positions → Sliding Window = contiguous range → Prefix Sum = cumulative state → Prefix Sum + Hashing = previous cumulative state lookup.

1. High-Level Comparison

These three patterns are closely related, but they solve **different kinds of array/string problems**. For Google DSA preparation, the important skill is not memorizing implementations—it is recognizing **which pattern fits the problem**.

Pattern	Core Idea	Typical Input	Main Goal	Typical Time	Typical Space
Two Pointers	Maintain two indices and move them based on condition	Sorted arrays, strings, and arrays	Find pairs, partition, compare elements	$O(n)$	$O(1)$
Sliding Window	Maintain a contiguous window of elements	Arrays [left, right] and expand/shrink	Find longest/shortest/contiguous window	$O(n)$	$O(1) - O(k)$
Prefix Sum	Store cumulative information	Arrays	Answer range/subarray aggregate processing efficiently	$O(n)$	$O(n)$
Prefix Sum + Hashing	Use cumulative state + hash to solve subarray conditions	Arrays with subarray conditions	Find subarrays satisfying condition	$O(n)$	$O(n)$

The most important distinction:

Two Pointers controls two positions. Sliding Window controls a contiguous range. Prefix Sum remembers cumulative information.

2. Two Pointers

Core Concept

You maintain two indices:

```

left
↓
[ ... array ... ]
      ↑
      right
  
```

Then move one or both pointers according to the problem's condition.

```

left → → →
[1, 2, 3, 4, 6]
      ← ← ← right
  
```

The pointers may move:

```
left++
```

or:

```
right--
```

or both.

3. When Should You Think Two Pointers?

Strong signals:

- sorted array
- find pair
- find two numbers
- target sum
- remove duplicates
- partition
- compare from both ends

- palindrome
- reverse
- merge-like traversal

Example:

Find two numbers in a sorted array whose sum equals target.

Think:

Two Pointers

4. Classic Two-Pointer Example

```
def two_sum_sorted(nums, target):
    left = 0
    right = len(nums) - 1

    while left < right:
        total = nums[left] + nums[right]

        if total == target:
            return [left, right]

        if total < target:
            left += 1
        else:
            right -= 1

    return []
```

For:

```
[1, 2, 4, 6, 8, 9]
target = 10
```

Start:

```
1 + 9 = 10
```

Done.

5. Why Does Two Pointers Work Here?

The array is **sorted**.

If:

```
nums[left] + nums[right] < target
```

then increasing `right` is not useful because `right` is already the largest value.

So we move:

```
left++
```

Similarly, if:

```
sum > target
```

we decrease:

```
right--
```

The sorted property gives us the ability to eliminate possibilities.

That's an important interview insight:

Two pointers often works because the structure of the input allows us to safely eliminate candidates.

6. Sliding Window

Sliding Window is actually a specialized form of pointer-based traversal.

You maintain:

[left right]

representing a **contiguous window**.

Example:

```
[2, 1, 5, 1, 3, 2]
  ↑       ↑
left    right
```

The window changes by:

expand → right++

and/or:

shrink → left++

7. When Should You Think Sliding Window?

Strong signals:

- contiguous subarray
- substring
- longest
- shortest/minimum
- maximum
- window
- consecutive elements
- at most K
- at least K
- distinct characters
- frequency within a range

Examples:

Longest substring without repeating characters.

Think:

Sliding Window

Or:

Minimum size subarray with sum ≥ target.

Think:

Sliding Window

when the input conditions support the necessary monotonic behavior.

8. Classic Sliding Window

Suppose:

```
nums = [2, 3, 1, 2, 4, 3]
target = 7
```

We want the minimum-length contiguous subarray whose sum is at least 7.

Conceptually:

```
[2, 3, 1, 2, 4, 3]
  ↑
  left
  ↑
  right
```

Expand:

```
[2]
```

then:

```
[2, 3]
```

then:

```
[2, 3, 1]
```

then:

```
[2, 3, 1, 2]
```

Once the condition is satisfied:

```
sum >= target
```

shrink from the left:

```
[3, 1, 2]
```

then:

```
[1, 2, 4]
```

and continue.

9. Sliding Window's Core Mental Model

```
EXPAND
↓
condition not satisfied
↓
EXPAND MORE
↓
condition satisfied
↓
SHRINK
↓
try to improve answer
```

So:

```
right → expands
```

left → shrinks

This is different from generic Two Pointers.

10. Important Relationship

You can think of:

```
Two Pointers
  ↓
Two indices
  ↓
One common specialization
  ↓
Sliding Window
```

But don't treat them as identical.

Two Pointers

The two pointers don't necessarily represent a window.

Example:

left → ← right

for palindrome checking.

There isn't necessarily a meaningful "window aggregate."

Sliding Window

The pointers specifically represent:

[left, right]

a **contiguous range**.

11. Prefix Sum Basics

Prefix Sum uses a completely different idea.

Instead of maintaining a moving window, we calculate cumulative information.

Given:

```
nums = [2, 4, 1, 3]
```

we create:

```
prefix = [0, 2, 6, 7, 10]
```

Meaning:

```
prefix[0] = 0
```

```
prefix[1] = 2
```

```
prefix[2] = 2 + 4 = 6
```

```
prefix[3] = 2 + 4 + 1 = 7
```

```
prefix[4] = 2 + 4 + 1 + 3 = 10
```

12. Why Prefix Sum?

Suppose you repeatedly ask:

What is the sum from index `1` to `3`?

Without prefix sums:

```
nums[1] + nums[2] + nums[3]
```

Potentially:

$O(n)$

With prefix sums:

```
prefix[4] - prefix[1]
=
10 - 2
=
8
```

The range sum becomes:

$O(1)$

after:

$O(n)$

preprocessing.

13. Prefix Sum Mental Model

```
Array
↓
Cumulative state
↓
Prefix array
↓
Range operation
↓
Difference between two prefix states
```

The fundamental equation is:

RangeSum = Prefix[right+1] - Prefix[left]

14. Prefix Sum + Hashing

This is where prefix sum becomes significantly more powerful.

Suppose the problem asks:

Count subarrays whose sum equals K.

We know:

```
currentPrefix - previousPrefix = K
```

Therefore:

```
previousPrefix = currentPrefix - K
```

So maintain:

```
HashMap:
prefix_sum → frequency
```

For every new prefix:

```
required = current_prefix - K
```

Look it up.

This gives:

```
O(n) average time
```

15. The Biggest Difference

Consider this problem:

Find the number of subarrays with sum exactly K.

Two Pointers?

Usually:

■

especially if negative numbers are possible.

Sliding Window?

Usually:

■

because negative numbers break the required monotonic behavior.

Prefix Sum + Hashing?

■

This is the natural pattern.

16. Why Sliding Window Fails With Negative Numbers

Consider:

```
[3, -2, 1, -1]
```

Suppose your window sum is too large.

Can you safely shrink?

Not necessarily.

Removing:

```
3
```

decreases the sum.

But removing:

```
-2
```

would **increase** the sum.

Therefore there is no simple monotonic relationship.

Prefix Sum doesn't have this problem.

It simply records:

```
current prefix
```

and asks:

Did I see current - K?

17. Side-by-Side Mental Models

Two Pointers

left → ← right

Use positions to eliminate candidates.

Sliding Window

left → [elements] ← right

Use a moving contiguous range.

Prefix Sum

0 → P1 → P2 → P3 → P4

Use cumulative state.

Prefix Sum + Hashing

Current prefix
↓
Current - K
↓
HashMap lookup
↓
Previous prefix
↓
Valid subarray

18. Comparison by Problem Type

Problem Type	Two Pointers	Sliding Window	Prefix Sum
Pair sum in sorted array	■■■	■	Possible but unnecessary
Palindrome	■■■	■	■
Remove duplicates	■■■	■	■
Reverse array/string	■■■	■	■
Longest substring	Sometimes	■■■	Sometimes
Minimum valid subarray	Sometimes	■■■	Sometimes
Range sum queries	■	■	■■■
Exact subarray sum K	■	Usually ■	■■■
Negative numbers + exact sum	■	Usually ■	■■■
Zero-sum subarray	■	Usually ■	■■■
Equal 0s and 1s	■	Usually ■	■■■
Sum divisible by K	■	■	■■■

19. Two Pointers vs Sliding Window

This is one of the most important distinctions for you.

Two Pointers

The pointers may represent **two independent positions**.

```
[1, 2, 3, 4, 6]
  ↑       ↑
left      right
```

Goal:

Find pair

Sliding Window

The pointers represent **boundaries of one contiguous region**.

```
[1, 2, 3, 4, 6]
  ↑-----↑
left      right
```

Goal:

Analyze this window.

Therefore:

Every sliding-window solution uses two boundary pointers, but not every two-pointer solution is a sliding-window solution.

20. Sliding Window vs Prefix Sum

These can look deceptively similar because both can solve subarray problems.

But their mechanisms are different.

Sliding Window

You maintain the current range:

```
[left, right]
```

and dynamically change it.

Prefix Sum

You don't need to maintain a valid current range.

Instead:

```
prefix[j] - prefix[i]
```

tells you the sum of a range.

21. Example Comparison

Suppose:

```
nums = [2, 3, 1, 2, 4, 3]
```

and:

```
target = 7
```

Problem A

Minimum length subarray with sum ≥ 7 .

Potential approach:

Sliding Window

because with positive numbers:

expand → sum increases
shrink → sum decreases

This gives useful monotonic behavior.

Problem B

Count subarrays whose sum is exactly 7.

Think:

Prefix Sum + Hashing

especially when arbitrary integers/negative numbers are possible.

22. Prefix Sum vs Sliding Window: Key Question

Do I need to maintain a valid window dynamically, or do I need to compare cumulative states?

If:

maintain window

→ Sliding Window.

If:

compare prefix states

→ Prefix Sum.

23. Two Pointers vs Prefix Sum

Suppose:

Find two numbers that add to target in a sorted array.

Two Pointers:

$O(n)$
 $O(1)$

Prefix Sum:

Not the natural solution.

Why?

Because the problem isn't about a contiguous range.

It's about:

two elements

This is a major clue.

24. A Critical Recognition Rule

Before selecting an algorithm, ask:

Question 1

Is the problem about a **pair of elements**?

→ Two Pointers

especially if sorted.

Question 2

Is it about a **contiguous range** that must dynamically satisfy a condition?

→ Sliding Window

Question 3

Is it about a **range sum / cumulative state**?

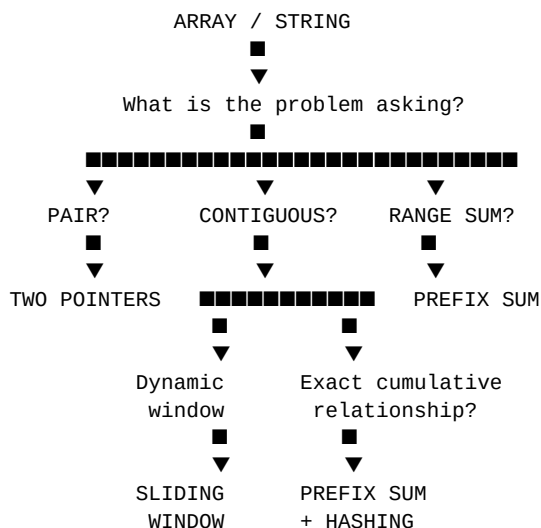
→ Prefix Sum

Question 4

Do I need to find previous cumulative states?

→ Prefix Sum + Hashing

25. Decision Tree



26. Pattern Recognition Table

Signal in Problem	Think
Sorted array + pair	Two Pointers
Find pair with target	Two Pointers
Compare left/right	Two Pointers
Palindrome	Two Pointers
Contiguous substring	Sliding Window
Longest substring	Sliding Window
Shortest valid subarray	Sliding Window
At most K	Sliding Window
Exactly K subarray sum	Prefix Sum + Hashing
Range sum queries	Prefix Sum
Zero-sum subarray	Prefix Sum + Hashing
Longest subarray sum K	Prefix Sum + Hashing
Negative numbers + exact sum	Prefix Sum + Hashing
Sum divisible by K	Prefix Sum + Hashing

27. Complexity Comparison

Two Pointers

Usually:

Time: $O(n)$
Space: $O(1)$

because each pointer moves through the array a limited number of times.

Sliding Window

Usually:

Time: $O(n)$
Space: $O(1)$

or:

$O(k)$

if maintaining a frequency map/set.

Important:

Even though there is sometimes a nested-looking structure:

```
while right < n:
    ...
    while condition:
        left += 1
```

the complexity can still be:

$O(n)$

because `left` and `right` each move forward at most `n` times.

Prefix Sum

Preprocessing:

$O(n)$

Each range query:

$O(1)$

Space:

$O(n)$

Prefix Sum + Hashing

Usually:

Time: $O(n)$ average
Space: $O(n)$

because every prefix sum may need to be stored.

28. Space Trade-Off

This is another important interview distinction.

Two Pointers

Usually:

$O(1)$

extra space.

Sliding Window

Usually:

$O(1)$

or:

$O(k)$

if a frequency map is required.

Prefix Sum

Usually:

$O(n)$

for the prefix array.

Prefix Sum + Hashing

Usually:

$O(n)$

for the hash map.

So:

Prefix-based approaches often trade memory for faster range/subarray reasoning.

29. The Three Patterns on One Array

Take:

[1, 2, 3, 4, 5]

Two Pointers

Could examine:

left $\rightarrow$ 1
right $\rightarrow$ 5

and search for a pair.

Sliding Window

Could maintain:

[2, 3, 4]
↑ ↑
left right

and expand/shrink based on a condition.

Prefix Sum

Build:

[0, 1, 3, 6, 10, 15]

and answer range sums through subtraction.

Same array.

Completely different mental models.

30. Common Interview Trap

Problem:

Find the number of subarrays with sum K.

A candidate may say:

"It's a sliding-window problem because it's a subarray."

That is incomplete reasoning.

The correct question is:

Does the window have the monotonic property required by sliding window?

If negative values are allowed:

Prefix Sum + Hashing

is usually the safer/general solution.

31. Another Interview Trap

Problem:

Find two numbers that sum to target.

A candidate might immediately use a HashMap.

That can work for an unsorted array:

$O(n)$ time
 $O(n)$ space

But if the array is sorted:

Two Pointers
 $O(n)$ time
 $O(1)$ space

is often better.

This is where interviewers may ask:

"Can you optimize the space?"

And you should recognize:

Sorted $\rightarrow$ Two Pointers

32. Another Important Distinction

Prefix Sum can answer:

What is the sum of this range?

But it does not necessarily tell you:

Is this the smallest valid range?

For optimization problems involving a dynamically changing valid range, Sliding Window may be more natural.

So don't think:

"Subarray" → Prefix Sum

Instead think:

What relationship am I trying to exploit?

33. Interview Pattern Recognition Framework

When you see an array/string problem, run this checklist.

Step 1 — Identify structure

Is it:

array?
string?
sorted?
unsorted?
positive?
negative?

Step 2 — Identify relationship

Is it asking about:

pair?
range?
subarray?
substring?
frequency?
sum?
maximum?
minimum?

Step 3 — Look for constraints

Especially:

$n \leq ?$
negative values?
sorted?
number of queries?

Step 4 — Choose pattern

Pair + sorted
→ Two Pointers

Contiguous + dynamic constraint
→ Sliding Window

Range aggregate
→ Prefix Sum

Exact subarray sum + arbitrary integers
→ Prefix Sum + Hashing

34. Google Interview-Level Comparison

You should be able to explain the differences verbally:

Two Pointers

"I use two indices to exploit ordering or structural relationships and eliminate candidates without revisiting them."

Sliding Window

"I maintain a contiguous interval using left and right boundaries, expanding and shrinking it while maintaining a window invariant."

Prefix Sum

"I precompute cumulative sums so a range sum can be obtained as the difference between two prefix sums."

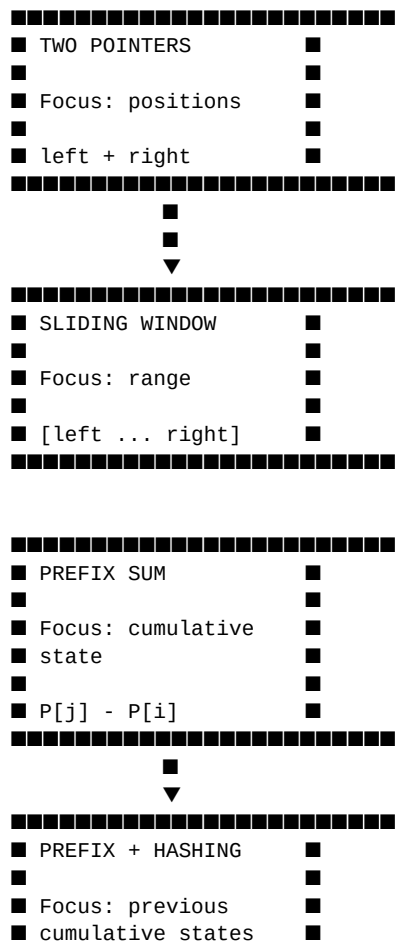
Prefix Sum + Hashing

"I maintain cumulative sums and use a hash map to efficiently find previous prefix states that satisfy the required difference."

These explanations demonstrate understanding rather than memorization.

35. The Most Important Comparison

Think of the patterns this way:





36. Your Pattern Recognition Cheat Sheet

Memorize **this**, not the code.

PAIR?



TWO POINTERS

CONTIGUOUS RANGE?



Does the condition support expand/shrink?



YES → SLIDING WINDOW

NO / exact cumulative relationship → PREFIX SUM

RANGE SUM?



PREFIX SUM

SUBARRAY SUM = K?



PREFIX SUM + HASHING

LONGEST SUBARRAY SUM = K?



PREFIX SUM + HASHING

prefix → earliest index

COUNT SUBARRAYS SUM = K?



PREFIX SUM + HASHING

prefix → frequency

ZERO-SUM SUBARRAY?



REPEATED PREFIX SUM

DIVISIBLE BY K?



PREFIX SUM % K + HASHING

prefix → frequency

37. The 4 questions to ask yourself in a Google interview

When you see an array problem, ask:

1. Am I finding a pair using ordering?

→ **Two Pointers**

2. Am I maintaining a contiguous range whose validity changes as I expand/shrink it?

→ **Sliding Window**

3. Am I repeatedly asking for a range sum/aggregate?

→ **Prefix Sum**

→ Prefix Sum + Hashing

Question	Two Pointers	Sliding Window	Prefix Sum	Prefix + Hashing
What does it track?	Two positions	A contiguous window	Cumulative state	Cumulative state + previous states
Requires contiguous range?	No	Yes	Often	Usually
Requires sorted input?	Often, but not always	No	No	No
Handles negatives?	Depends	Often problematic for sum constraints	Yes	Yes
Pair problems?	Excellent	No	Usually unnecessary	Usually unnecessary
Range sums?	No	No	Excellent	Possible
Exact subarray sum?	Usually no	Limited	Possible	Excellent
Longest valid subarray?	Sometimes	Excellent for suitable constraints	Excellent for sum-K variants	Excellent
Count subarrays?	Rarely	Sometimes	Sometimes	Excellent
Typical space	$O(1)$	$O(1)$ – $O(k)$	$O(n)$	$O(n)$
Core question	"Which pointer moves?"	"Should I expand or shrink?"	"What are the cumulative states?"	"What previous state do I need?"

■ current - K

The most important distinction to internalize is:

Two Pointers = positions → Sliding Window = contiguous range → Prefix Sum = cumulative state → Prefix Sum + Hashing = previous cumulative state lookup.

That progression is exactly what you should use for **Day 5 DSA pattern recognition**.